

WEEK – 8

Experiment No. 8: Interfaces

Title

Employee Salary Details Using Interfaces

Objective / Aim

To write a Java program using **interfaces** to calculate and display the monthly salary details of different categories of employees, such as Regular Employees and Contract Employees.

Problem Statement

A software company has different categories of employees, including:

- Regular Employees
- Contract Employees
- Vendors

The payroll structure differs for Regular and Contract Employees.

Regular Employee:

- Basic Pay = Rs.25,000
- HRA = Rs.15,000
- T.A = Rs.5,000

Contract Employee:

- Basic Pay = Rs.12,000
- HRA = Rs.0
- T.A = Rs.3,000

The program should accept an employee ID and identify the employee category. It should then calculate and display the corresponding monthly salary details.

Concepts Used

Concept	Description
Interface	Defines common salary-related methods
Interface Implementation	Employee classes implement the interface
Interface Reference	Interface reference accesses implementing objects
Method Overriding	Implementing classes provide method definitions
Encapsulation	Salary details are handled through class methods
Polymorphism	Different employee types can be accessed through the interface

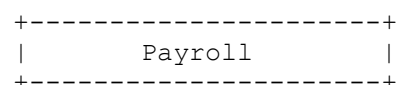
Salary Structure

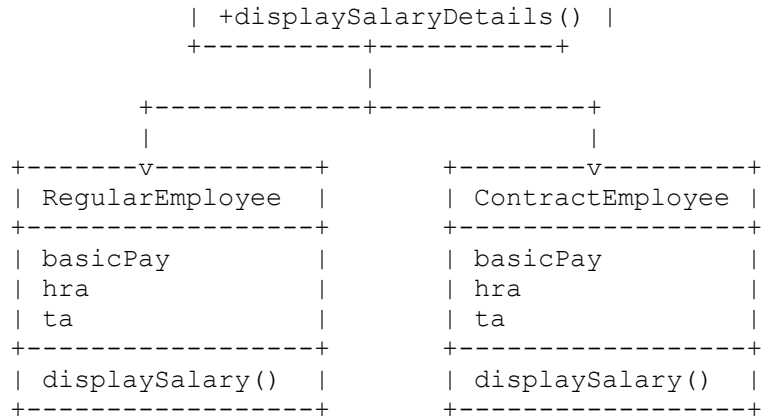
Employee Type	Basic Pay	HRA	T.A	Total
Regular Employee	Rs.25,000	Rs.15,000	Rs.5,000	Rs.45,000
Contract Employee	Rs.12,000	Rs.0	Rs.3,000	Rs.15,000

Algorithm / Logic

1. Define an interface named `Payroll`.
2. Declare a method `displaySalaryDetails()` in the interface.
3. Create a `RegularEmployee` class that implements `Payroll`.
4. Store the salary components of a regular employee.
5. Calculate the total salary.
6. Create a `ContractEmployee` class that implements `Payroll`.
7. Store the salary components of a contract employee.
8. Calculate the total salary.
9. Read the employee ID from the user.
10. If the ID starts with `R`, create a Regular Employee object.
11. If the ID starts with `C`, create a Contract Employee object.
12. Access the object using the `Payroll` interface reference.
13. Display the salary details.

UML / Class Diagram





Source Code

```

package recordprograms;

import java.util.Scanner;

interface Payroll {
    void displaySalaryDetails();
}

class RegularEmployee implements Payroll {

    String employeeId;
    double basicPay = 25000;
    double hra = 15000;
    double ta = 5000;

    RegularEmployee(String employeeId) {
        this.employeeId = employeeId;
    }

    @Override
    public void displaySalaryDetails() {

        double total = basicPay + hra + ta;

        System.out.println("Salary Details:");
        System.out.println("Employee Id: " + employeeId);
        System.out.println("Basic Pay: " + basicPay);
        System.out.println("HRA: " + hra);
        System.out.println("T.A: " + ta);
        System.out.println("Total Amount: " + total);
    }
}

class ContractEmployee implements Payroll {

    String employeeId;
    double basicPay = 12000;
    double hra = 0;
    double ta = 3000;

```

```

ContractEmployee(String employeeId) {
    this.employeeId = employeeId;
}

@Override
public void displaySalaryDetails() {

    double total = basicPay + hra + ta;

    System.out.println("Salary Details:");
    System.out.println("Employee Id: " + employeeId);
    System.out.println("Basic Pay: " + basicPay);
    System.out.println("HRA: " + hra);
    System.out.println("T.A: " + ta);
    System.out.println("Total Amount: " + total);
}
}

public class EmployeePayroll {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Employee Id: ");
        String employeeId = sc.nextLine().trim().toUpperCase();

        Payroll employee;

        if (employeeId.startsWith("R")) {
            employee = new RegularEmployee(employeeId);
        }
        else if (employeeId.startsWith("C")) {
            employee = new ContractEmployee(employeeId);
        }
        else {
            System.out.println("Invalid Employee Id.");
            sc.close();
            return;
        }

        employee.displaySalaryDetails();

        sc.close();
    }
}

```

Sample Output

Test Case 1 – Regular Employee

```

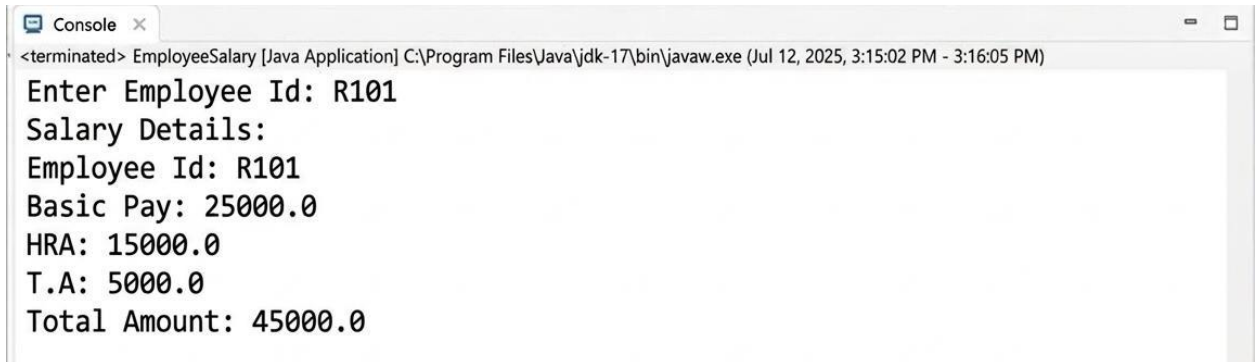
Enter Employee Id: R101
Salary Details:
Employee Id: R101
Basic Pay: 25000.0
HRA: 15000.0
T.A: 5000.0

```

Total Amount: 45000.0

Test Case 2 – Contract Employee

Enter Employee Id: C101
Salary Details:
Employee Id: C101
Basic Pay: 12000.0
HRA: 0.0
T.A: 3000.0
Total Amount: 15000.0



```
<terminated> EmployeeSalary [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (Jul 12, 2025, 3:15:02 PM - 3:16:05 PM)
Enter Employee Id: R101
Salary Details:
Employee Id: R101
Basic Pay: 25000.0
HRA: 15000.0
T.A: 5000.0
Total Amount: 45000.0
```

```
Enter Employee Id: C101
Salary Details:
Employee Id: C101
Employee Id: C101
Basic Pay: 12000.0
HRA: 0.0
T.A: 3000.0
Total Amount: 15000.0
```

Interface Reference Demonstration

The following statement demonstrates accessing an implementation through an interface reference:

```
Payroll employee;
```

The reference can point to different implementing classes:

```
employee = new RegularEmployee(employeeId);
```

or:

```
employee = new ContractEmployee(employeeId);
```

The same interface method is then called:

```
employee.displaySalaryDetails();
```

Java executes the appropriate implementation based on the actual object.

Compilation and Execution

If the program contains:

```
package recordprograms;
```

save the file as:

```
recordprograms/EmployeePayroll.java
```

Compile:

```
javac recordprograms/EmployeePayroll.java
```

Run:

```
java recordprograms.EmployeePayroll
```

Result

Thus, the Java program successfully calculates and displays the monthly salary details of Regular and Contract Employees using an **interface and interface references**.

Conclusion

The program demonstrates how an interface can define a common payroll operation while different employee classes provide their own salary implementations. It also demonstrates **interface implementation, method overriding, interface references, and polymorphism**.